

Lab 3C: CI/CD - Claude in Your Pipeline

Duration: 35 min

After: Production & CI/CD lecture (Day 3, Block 3)

Prerequisites: Lab 3B complete; GitHub account + `gh` CLI for Phase 2 (Phase 1 needs no GitHub)

Objective

Run Claude Code headless with `claude -p` under explicit `--max-turns` and `--allowedTools` budgets, parse structured JSON output, then wire the official `anthropics/claude-code-action@v1` GitHub Action for **advisory** PR review.

Deliverable (toolkit chain 11 of 12)

A headless review script (`scripts/ci-review.sh`) and a `.github/workflows/claude-review.yml` advisory workflow in the toolkit repo.

START MARKER: Regular terminal (NOT inside Claude), `cd business-dashboard`.

Phase 1: Headless Mode (15 min)

Headless = one prompt in, output out, exit code, no conversation. This is Claude as a tool other tools can call.

Step 1 - basic invocation:

```
claude -p "List all JS files in src/ and describe each in one sentence." --allowedTools
"Read,Glob"
```

Expected output marker: a file list prints, then the process exits - no interactive prompt returns.

Two flags you must never omit in CI:

`--allowedTools "Read,Grep,Glob"` - headless has no human to approve tools; without this Claude can't read files (or with looser settings could do too much).

`--max-turns 10` - caps the agentic loop so a confused run can't burn tokens forever.

Step 2 - pipe input:

```
git log --oneline -10 | claude -p "Generate a markdown changelog from this git log.
Sections: Features, Fixes, Chores." --max-turns 3
```

Expected output marker: a formatted changelog on stdout.

Step 3 - structured JSON:

```
claude -p "Review src/routes/tasks.js. Return JSON:
{ \"issues\": [{ \"severity\": \"high|medium|low\", \"message\": \"...\" }], \"score\": number }" \
--allowedTools "Read,Grep,Glob" --max-turns 10 --output-format json > review.json
cat review.json
```

Expected output marker: a JSON envelope with `type`, `result`, `session_id`, `total_cost_usd`, `duration_ms`, `num_turns`. Your review text is a string inside `.result`:

```
jq -r '.result' review.json
jq -r '.total_cost_usd' review.json
```

Step 4 - the reusable gate script. Create `scripts/ci-review.sh`:

```
#!/bin/bash
set -e
claude -p "Review the code in src/ for security and quality issues. Return JSON: {\"issues\":[{\\\"severity\\\":\\\"critical|high|medium|low\\\",\\\"message\\\":\\\"...\\\"}],\\\"score\\\":number}\" \\
--allowedTools "Read,Grep,Glob" \\
--max-turns 10 \\
--output-format json > review.json

SCORE=$(jq -r '.result' review.json | jq -r '.score // 0')
echo "Review score: $SCORE"
echo "Cost: \\$(jq -r '.total_cost_usd' review.json)"
if (( $(echo "$SCORE < 6" | bc -l) )); then
echo "REVIEW FAILED (score < 6)"; exit 1
fi
echo "REVIEW PASSED"
```

```
chmod +x scripts/ci-review.sh && ./scripts/ci-review.sh
```

Expected output marker: `Review score: <n>`, a cost line, and `REVIEW PASSED` (exit 0) - or a non-zero exit you can gate on. This exact script is your capstone CI check tomorrow. It works in ANY CI system: GitHub, GitLab, Azure, Jenkins.

***Billing note:** headless/SDK usage draws from a separate **weekly token pool** on paid plans - not your interactive budget, and not a separate monthly credit (verify at delivery - fast-moving).*

Phase 2: GitHub Action - Official, Advisory (15 min)

No GitHub access? Phase 1 was the core learning. Read this phase as reference - the instructor will demo.

Step 1 - repo + secret:

```
gh repo create business-dashboard --private --source=. --remote=origin --push
gh secret set ANTHROPIC_API_KEY # paste key when prompted
```

Step 2 - the official Action. Create `.github/workflows/claude-review.yml`:

```

name: Claude PR Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
      issues: read
    steps:
      - uses: actions/checkout@v4
    with:
      fetch-depth: 0
      - uses: anthropics/claude-code-action@v1
    with:
      anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
      prompt: |
        Review this pull request for security, quality, and test coverage.
        Post findings as a PR comment with severity labels.
        This review is ADVISORY - do not fail the check for findings.
      claude_args: "--max-turns 10 --allowedTools Read,Grep,Glob"

```

Note what you did NOT write: no install step, no JSON plumbing, no comment-posting script - the official action handles it.

Step 3 - trigger with a deliberately bad PR:

```

git checkout -b test-review
cat > src/bad-code.js <<'EOF'
const API_KEY = "sk-1234567890abcdef";
function runQuery(userInput) {
  return `SELECT * FROM users WHERE name = '${userInput}'`;
}
module.exports = { runQuery };
EOF
git add -A && git commit -m "test: add review-bait file"
git push -u origin test-review
gh pr create --title "Test: Claude advisory review" --body "Testing the review workflow"

```

Step 4 - watch:

```
gh run watch
```

Expected output marker: the workflow runs and a Claude comment appears on the PR flagging the hardcoded secret and SQL injection with severities. The check stays green - **advisory** informs, never blocks:

```
gh pr view --comments
```

Step 5 - verify advisory, then clean up. The merge button stays available despite critical findings - that's the advisory contract. Close without merging:

```
gh pr close test-review --delete-branch
git checkout main
```

Advisory vs gating: start advisory; graduate to gating (exit-code enforcement like Phase 1's script + branch protection) only after the team calibrates thresholds. Too strict too early = developers route around the bot.

FINISH MARKER - Success Checklist

- [] `claude -p` ran with BOTH `--allowedTools` and `--max-turns`
- [] JSON envelope parsed: `.result`, `.total_cost_usd`
- [] `scripts/ci-review.sh` runs and exits 0/1 on a score threshold
- [] `anthropics/claude-code-action@v1` workflow committed
- [] Advisory PR comment received on the bad-code PR (or watched in demo)
- [] You can state when to move advisory -> gating

If Stuck

Problem	Recovery
Headless can't read files	You omitted <code>--allowedTools</code>
<code>jq</code> errors on <code>review.json</code>	The response isn't bare JSON - extract <code>.result</code> first with <code>jq -r '.result'</code>
Action fails at auth	<code>gh secret list</code> must show <code>ANTHROPIC_API_KEY</code> ; re-set it
No PR comment appears	Check permissions: <code>pull-requests: write</code> in the workflow
<code>bc</code> not installed	Ask Claude to rewrite the threshold check as an integer compare
Runs slow/expensive	Lower <code>--max-turns</code> ; scope the prompt to changed files only

Debrief Questions

1. What do `--max-turns` and `--allowedTools` each protect you from in CI?
2. Why start advisory instead of gating on day one?
3. Which token pool does a CI run draw from, and why should your platform team care?